Master Context: Building "mygit" (A C++ Git Clone)

Context for AI: Act as my senior systems-engineering mentor. We are building a lightweight version control system from scratch in C++, heavily inspired by Git's internal architecture. I want to build this iteratively, focusing on clean C++ concepts (STL, OOP, `<filesystem>`) and low-level system design. Please guide me sprint by sprint.

1. Architectural Core (The Object Model)

Git is fundamentally a content-addressable filesystem that stores data as a Directed Acyclic Graph (DAG). We will implement the three core Git objects:

- **Blobs (Binary Large Objects):** Store only the *content* of a file, not its name. Addressed by the SHA-1 hash of the content.
- **Trees:** Represent directories. They map file names and permissions to Blob hashes or other Tree hashes.
- **Commits:** Snapshots of the project. They contain a pointer to the root Tree, a pointer to the parent Commit, a timestamp, author metadata, and a commit message.

2. Directory Structure

Our application will create and manage a hidden directory at the root of the user's project:

```
.mygit/
├── HEAD            # Points to the current active branch (e.g., ref: refs/hea
├── objects/        # The content-addressable store (folders named with first
└── refs/
    └── heads/      # Pointers to the latest commit of each branch
```

3. C++ Technical Requirements

- **Language Standards:** C++17 or C++20 (specifically for the `<filesystem>` library).
- **Standard Library:** `<fstream>` for I/O, `<string>`, `<vector>`, `<unordered_map>`.
- **Hashing:** We will need a basic SHA-1 hashing utility function (can use OpenSSL or a lightweight standalone C++ header).
- **Compression:** For the MVP, we can store raw text in the `objects` folder. *Advanced phase: zlib compression.*

4. The Roadmap (Sprint Breakdown)

Sprint 1: The Foundation & The Blob

- `mygit init`: Create the `.mygit` directory structure safely.
- `mygit hash-object <file>`: Read a file, compute its SHA-1 hash, and write it to `.mygit/objects/`.
- `mygit cat-file <hash>`: Read and decompress a blob from the object store back into readable text.

Sprint 2: Trees and Commits

- `mygit write-tree` : Recursively traverse the current working directory, hash the files (blobs), and write Tree objects to the store.
- `mygit commit -m "Message"` : Create a Commit object pointing to the current Tree and the parent Commit, then update the branch `ref` .

Sprint 3: History & Time Travel

- `mygit log` : Traverse the commit parent pointers backward to print the history.
- `mygit checkout <hash>` : Parse a historical Tree object and safely overwrite the working directory to match that snapshot.

User command to start: "I am ready. Let's start with Sprint 1: setting up the C++ project structure and writing the `mygit init` command."